Інсталяція та налаштування AgroCD

Що таке Argo CD?

Argo CD — це потужний **GitOps-контролер для Kubernetes**, який дозволяє автоматичн розгортати ваші застосунки на основі змін у Git-репозиторії.

Argo CD:

- Постійно відстежує Git-репозиторій з маніфестами;
- Синхронізує кластер із бажаним станом;
- Дає зручний вебінтерфейс, CLI та API для керування розгортаннями;
- Підтримує **Helm**, **Kustomize**, **Ksonnet**, **Jsonnet** і plain YAML;
- Дозволяє повністю автоматизувати оновлення застосунків і відкотитися до попередньої версії в один клік.

Це один з найпопулярніших інструментів для **реалізації GitOps у Kubernetes**, особливо великих DevOps-командах або масштабних проєктах.

Основні можливості Argo CD:

- **Синхронізація стану**

Argo порівнює бажаний стан у Git і фактичний у кластері та автоматично приводить їх до відповідності.

- **Підтримка шаблонізаторів**

Працює з Helm-чартами, Kustomize, YAML — що завгодно.

- **Зручна візуалізація**

У вебінтерфейсі видно статуси застосунків, синхронізацію, хелс-чеки, історію змін.

- **RBAC, багатокористувацький режим, SSO**

Підтримка enterprise-фіч для захищених і командних середовищ.

- **Масштабування**

Підтримка десятків або сотень застосунків, організація через `projects`, `applications` тощо.

Тож у нас уже є готова структура модулів для кластера, давайте в нього додамо новий модуль.

Доповнимо нашу структуру проєкту наступним:

```
Progect/
│
├── main.tf                    # Головний файл для підключення
модулів
├── backend.tf                 # Налаштування бекенду для стейтів (S3
+ DynamoDB
├── outputs.tf                 # Загальні виводи ресурсів
│
├── modules/                   # Каталог з усіма модулями
│   ├── s3-backend/            # Модуль для S3 та DynamoDB
│   │   ├── s3.tf              # Створення S3-бакета
│   │   ├── dynamodb.tf        # Створення DynamoDB
│   │   ├── variables.tf       # Змінні для S3
│   │   └── outputs.tf         # Виведення інформації про S3 та
DynamoDB
│   │
│   ├── vpc/                   # Модуль для VPC
│   │   ├── vpc.tf             # Створення VPC, підмереж, Internet
Gateway
│   │   ├── routes.tf          # Налаштування маршрутизації
│   │   ├── variables.tf       # Змінні для VPC
│   │   └── outputs.tf
│   ├── ecr/                   # Модуль для ECR
│   │   ├── ecr.tf             # Створення ECR репозиторію
│   │   ├── variables.tf       # Змінні для ECR
│   │   └── outputs.tf         # Виведення URL репозиторію
│   │
│   ├── eks/                       # Модуль для Kubernetes кластера
│   │   ├── eks.tf                 # Створення кластера
│   │   ├── aws_ebs_csi_driver.tf # Встановлення плагіну csi drive
│   │   ├── variables.tf       # Змінні для EKS
│   │   └── outputs.tf         # Виведення інформації про кластер
│   │
│   ├── jenkins/               # Модуль для Helm-установки Jenkins
│   │   ├── jenkins.tf         # Helm release для Jenkins
│   │   ├── variables.tf       # Змінні (ресурси, креденшели, values)
│   │   ├── providers.tf       # Оголошення провайдерів
│   │   ├── values.yaml        # Конфігурація jenkins
│   │   └── outputs.tf         # Виводи (URL, пароль адміністратора)
```

```
│   │
│   └── argo_cd/            # ✅ Новий модуль для Helm-установки
Argo CD
│       ├── jenkins.tf      # Helm release для Jenkins
│       ├── variables.tf    # Змінні (версія чарта, namespace,
repo URL тощо)
│       ├── providers.tf    # Kubernetes+Helm.  переносимо з
модуля jenkins
│       ├── values.yaml     # Кастомна конфігурація Argo CD
│       ├── outputs.tf      # Виводи (hostname, initial admin
password)
│           └──charts/                  # Helm-чарт для створення
app'ів
│           ├── Chart.yaml
│           ├── values.yaml          # Список applications,
repositories
│               └── templates/
│               ├── application.yaml
│               └── repository.yaml
├── charts/
│   └── django-app/
│       ├── templates/
│       │   ├── deployment.yaml
│       │   ├── service.yaml
│       │   ├── configmap.yaml
│       │   └── hpa.yaml
│       ├── Chart.yaml
│       └── values.yaml     # ConfigMap зі змінними середовища
```

Поїхали створювати наш модуль для Helm 🚀

Файл: main.tf

Головний вхідний файл Terraform-проєкту. Тут підключається модуль ArgoCD і передаються необхідні параметри.

```
module "argo_cd" {
  source       = "./modules/argo-cd"
  namespace    = "argocd"
```

```
  chart_version = "5.46.4"
}
```

Щоб ArgoCD був не просто встановлений, а одразу був готовий до роботи з нашим кодом, створимо файл `modules/argo_cd/argo_cd.tf`:

```
resource "helm_release" "argo_cd" {
  name       = var.name
  namespace  = var.namespace
  repository = "<https://argoproj.github.io/argo-helm>"
  chart      = "argo-cd"
  version    = var.chart_version

  values = [
    file("${path.module}/values.yaml")
  ]

  create_namespace = true
}

resource "helm_release" "argo_apps" {
  name       = "${var.name}-apps"
  chart      = "${path.module}/charts"
  namespace  = var.namespace
  create_namespace = false

  values = [
    file("${path.module}/values.yaml")
  ]
  depends_on = [helm_release.argo_cd]
}
```

Пояснення:

- `helm_release.argo_cd` — встановлює **Argo CD**, інструмент GitOps для автоматичного деплою Kubernetes-ресурсів із Git-репозиторію.
- Використовується офіційний Helm-чарт із репозиторію `argoproj`.

`name` — ім’я Helm-релізу (наприклад, `argo-cd`). Це ім’я буде використано в назвах ресурсів, створених у кластері, як-от `argo-cd-server`.

`namespace` — Kubernetes namespace для встановлення Argo CD. Якщо він не існує, буде створений завдяки `create_namespace = true`.

`repository` — офіційне джерело Helm-чартів для Argo CD.

`chart` — назва чарта Argo CD (`argo-cd`), що буде використаний для інсталяції.

`version` — визначає конкретну версію чарта (наприклад, `"5.46.4"`), що забезпечу контроль за оновленням.

`values` — шлях до YAML-файлу, що містить кастомні параметри конфігурації (UI, ingress, RBAC, etc).

Argo CD Applications — це окремі застосунки (apps), якими Argo CD керує автоматично Простими словами:

Argo CD — це GitOps-інструмент, що стежить за Git-репозиторієм і автоматично **доставляє ваші YAML-и у кластер**.

Application — це як «проєкт», який описує:

звідки брати код (Git, Helm);

у який namespace деплоїти;

що саме запускати (наприклад, nginx або ваш бекенд).

Наприклад, якщо ви маєте бекенд, фронт і базу — кожен із них може бути окремим **Argo CD Application**.

`helm_release.argo_apps` — окремий Helm-реліз для **власних Argo CD Application** які описані як підчарти або через `Application` CRD у локальній директорії `${path.module}/charts`.

`chart` — шлях до локальної директорії з Helm-чартами (`charts`), яка містить визначення застосунків, що мають бути синхронізовані через Argo CD.

`depends_on` — забезпечує, щоб Argo CD був встановлений **перед** деплоєм застосунків.

`create_namespace = false` — очікує, що namespace уже існує (встановлений попереднім ресурсом).

`values` — знову використовує спільний конфігураційний файл `values.yaml`,

який може містити загальні параметри для всіх застосунків.

Цей ресурс є першим кроком у повному GitOps-ланцюгу: він встановлює Argo CD, який потім через окремий Helm-чарт (`argo-apps`) розгортає всі застосунки автоматично на основі Git-конфігів.

Також прив’яжемо Load Balancer сервісу нашого ArgoCD у файлі values.yaml:

```
server:
  service:
    type: LoadBalancer
    port: 443
```

Створюємо стандартні файли, які ми вже створювали в попередніх завданнях: outputs, providers та vatiables.

`outputs.tf`

```
output "argo_cd_server_service" {
  description = "Argo CD server service"
  value       = "argo-cd.${var.namespace}.svc.cluster.local"
}

output "admin_password" {
  description = "Initial admin password"
  value       = "Run: kubectl -n ${var.namespace} get secret
argocd-initial-admin-secret -o jsonpath={.data.password} | base64 -
d"
}
```

`variables.tf`

```
variable "name" {
  description = "Назва Helm-релізу"
  type        = string
  default     = "argo-cd"
}

variable "namespace" {
  description = "K8s namespace для Argo CD"
  type        = string
```

```
  default     = "argocd"
}

variable "chart_version" {
  description = "Версія Argo CD чарта"
  type        = string
  default     = "5.46.4"
}
```

providers.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 4.0"
    }
    helm = {
      source  = "hashicorp/helm"
      version = ">= 2.0.0"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = ">= 2.0.0"
    }
  }
}
```

Переходимо в директорію `modules/argo_cd/chart`, у якій опишемо чарт нашого **Argo CD Applications.**

Згадаємо, як ми створювали структуру чарта в темі 7 («Створення власного Helm-чарта для Django-проєкту»), та зробимо аналогічно для **Applications** 👇🏻

Згадати

Chart.yaml

```yaml
apiVersion: v2
name: argo-apps
version: 0.1.0
description: Deploy Argo CD Applications and Repositories
```

values.yaml

```yaml
applications:
  - name: example-app
    namespace: default
    project: default
    source:
      repoURL: <https://github.com/YOUR_USERNAME/example-repo.git>
      path: django-chart
      targetRevision: main
      helm:
        valueFiles:
          - values.yaml
    destination:
      server: <https://kubernetes.default.svc>
      namespace: default
    syncPolicy:
      automated:
        prune: true
        selfHeal: true

repositories:
  - name: example-app
    url: <https://github.com/YOUR_USERNAME/example-repo.git>
    username: YOUR_USERNAME
    password: YOUR_PASS

repoConfig:
  insecure: "true"
  enableLfs: "true"
```

applications

Цей блок описує список Argo CD застосунків, які потрібно створити. Кожен застосунок — це окремий деплой у кластері, яким керує Argo CD.

- `name` — ім’я застосунку в Argo CD. Використовується для ідентифікації в інтерфейс та API.
- `namespace` — namespace у Kubernetes, куди буде задеплоєний застосунок.
- `project` — назва Argo CD проєкту. Може використовуватись для ізоляції застосунків або керування доступом.
- `source` — джерело коду застосунку:
- `repoURL` — посилання на Git-репозиторій, де зберігаються Helm chart або Kubernetes маніфести.
- `path` — шлях усередині репозиторію до потрібного чарта або YAML-файлів.
- `targetRevision` — гілка або тег Git, з якого брати застосунок (наприклад, `main`).
- `helm.valueFiles` — перелік YAML-файлів із налаштуваннями, які будуть передані Helm-чарту.
- `destination` — вказує, куди саме деплоїти застосунок:
- `server` — адреса Kubernetes API-сервера. Для внутрішнього кластера використовується `https://kubernetes.default.svc`.
- `namespace` — Kubernetes namespace для цього конкретного застосунку.
- `syncPolicy` — політика синхронізації:
- `automated` — включає автоматичне застосування змін.
- `prune: true` — видаляти ресурси, які більше не описані у Git.
- `selfHeal: true` — автоматично виправляти відхилення від Git-стану, якщо щось змінилось вручну у кластері.

`repositories`

Список приватних Git-репозиторіїв, які Argo CD повинен знати й мати до них доступ.

- `name` — ім’я репозиторію для внутрішнього використання.
- `url` — повна URL-адреса Git-репозиторію.
- `username`, `password` — облікові дані для доступу до приватного репозиторію.

`repoConfig`

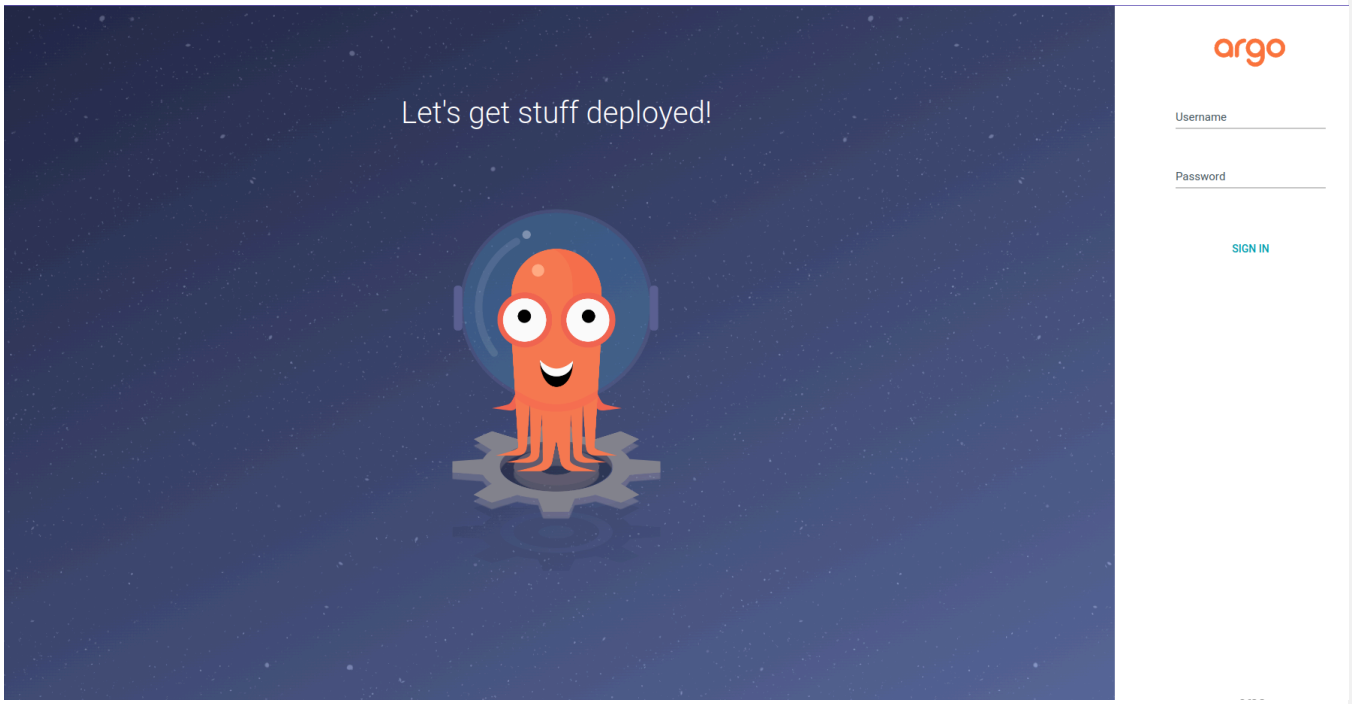
Конфігурація для репозиторіїв, доданих у `repositories`.

`insecure: "true"` — дозволяє підключення до репозиторію без перевірки SSL-сертифікатів. Не рекомендується для продакшену.

`enableLfs: "true"` — дозволяє використання Git LFS (Large File Storage), якщо в репозиторії зберігаються великі файли.

Запустимо Terraform із кореня проєкту й перевіримо ArgoCD:

```
terraform apply
```

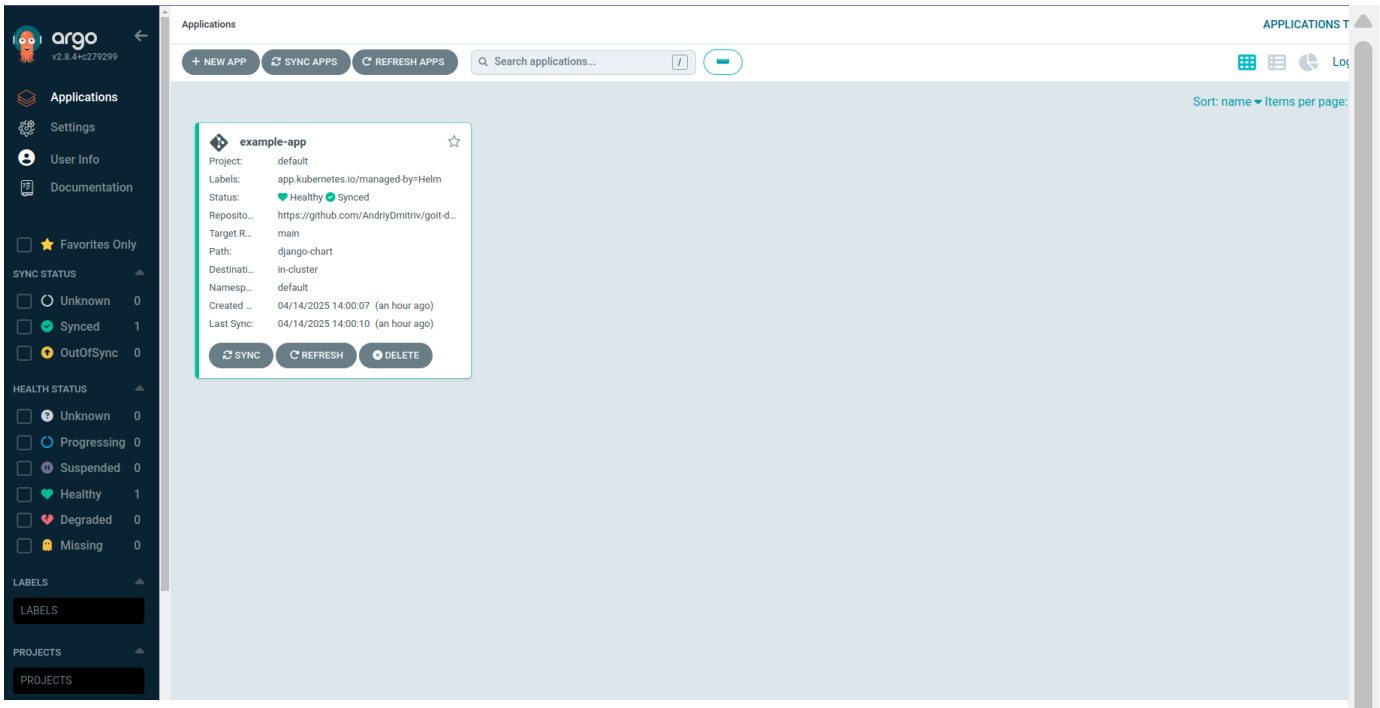


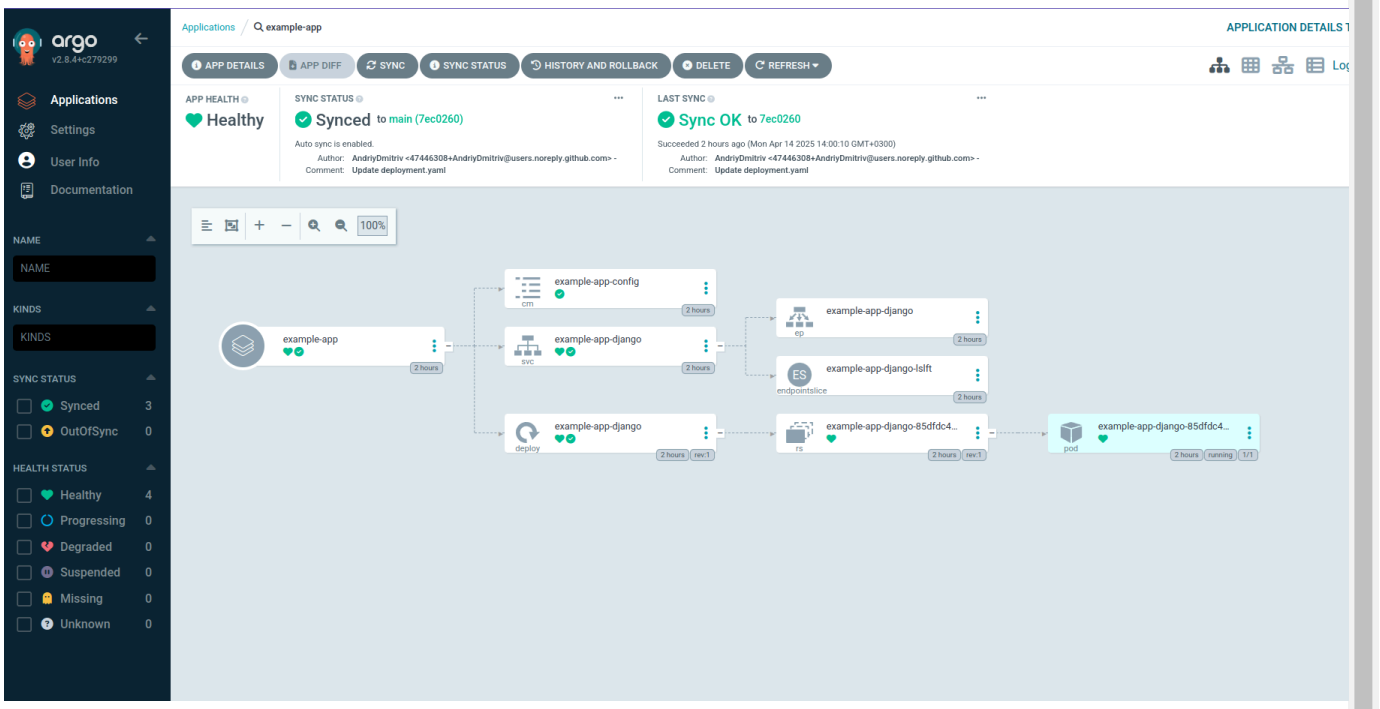
ArgoCD запущений. Для того щоб увійти, витягнемо пароль за допомогою команди:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
```

Користувач *admin*

Розгорнемо й поглянемо докладніше. Для цього натисніть на `applications` → `example.app` (на наведеному прикладі).





Коротко про те, що тут відбувається:

Застосунок `example-app` успішно синхронізований (`Synced`) і в хорошому стані (`Healthy`).

Він складається з кількох Kubernetes-ресурсів:

`ConfigMap` (`example-app-config`)

`Service` (`example-app-django`)

`Deployment` → `ReplicaSet` → `Pod` (усі з префіксом `example-app-django`)

Створення повноцінного CI/CD процесу

Після того як ми налаштували ArgoCD Applications у попередній підтемі та реалізували Jenkins Pipeline для автоматичного білду Docker-образу, ми фактично завершили 2 ключові етапи GitOps-підходу: збірку артефактів і керування розгортанням через ArgoCD.

Настав час зібрати все воєдино й інтегрувати ці процеси безпосередньо в сам репозиторій, додавши туди файл `Jenkinsfile`. Це забезпечить прозорий, версіонований і контрольований процес CI/CD, який реагує на зміни в коді або інфраструктурі. Таким чином, кожен пуш у репозиторій автоматично ініціює повний цикл: збірка, публікація, оновлення Helm chart і розгортання у кластер.

Це критично важливо для сучасної DevOps-практики, де швидкість і надійність змін мають вирішальне значення.

Тож до роботи 👇🏻

Доповнимо `Jenkinsfile`

```
pipeline {
  agent {
    kubernetes {
      yaml """
apiVersion: v1
kind: Pod
metadata:
  labels:
    some-label: jenkins-kaniko
spec:
  serviceAccountName: jenkins-sa
  containers:
    - name: kaniko
      image: gcr.io/kaniko-project/executor:v1.16.0-debug
      imagePullPolicy: Always
      command:
        - sleep
      args:
        - 99d
    - name: git
      image: alpine/git
      command:
```

```
        - sleep
      args:
        - 99d
"""
    }
  }

  environment {
    ECR_REGISTRY = "864004266599.dkr.ecr.us-west-2.amazonaws.com"
    IMAGE_NAME   = "app"
    IMAGE_TAG    = "v1.0.${BUILD_NUMBER}"

    COMMIT_EMAIL = "jenkins@localhost"
    COMMIT_NAME  = "jenkins"
  }

  stages {
    stage('Build & Push Docker Image') {
      steps {
        container('kaniko') {
          sh '''
            /kaniko/executor \\
              --context `pwd` \\
              --dockerfile `pwd`/Dockerfile \\
              --destination=$ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG \\
              --cache=true \\
              --insecure \\
              --skip-tls-verify
          '''
        }
      }
    }

    stage('Update Chart Tag in Git') {
      steps {
        container('git') {
          withCredentials([usernamePassword(credentialsId: 'github-
token', usernameVariable: 'GIT_USERNAME', passwordVariable:
'GIT_PAT')]) {
            sh '''
              git clone
<https://$GIT_USERNAME:$GIT_PAT@github.com/AndriyDmitriv/goit-
devops.git>
```

```
                cd goit-devops/django-chart

                sed -i "s/tag: .*/tag: $IMAGE_TAG/" values.yaml

                git config user.email "$COMMIT_EMAIL"
                git config user.name "$COMMIT_NAME"

                git add values.yaml
                git commit -m "Update image tag to $IMAGE_TAG"
                git push origin main
              '''
            }
          }
        }
      }
    }
  }
```

`stage('Update Chart Tag in Git')`

Цей етап оновлює файл `values.yaml` у Helm chart після збірки Docker-образу. Він змінює рядок `tag: ...` на нову версію образу (`$IMAGE_TAG`) і пушить зміни в репозиторій `goit-devops` .

Ключові кроки:

- **`container('git')`** — використовує Git-контейнер для виконання всіх Git-команд.
- **`withCredentials([...])`** — витягує логін і GitHub PAT (персональний токен) із Jenkins credentials із ID `github-token` .
- `usernameVariable: 'GIT_USERNAME'`
- `passwordVariable: 'GIT_PAT'`

Внутрішній shell-скрипт:

Клонує репозиторій із GitHub з використанням Jenkins credentials.

```
git clone
<https://$GIT_USERNAME:$GIT_PAT@github.com/AndriyDmitriv/goit-
```

```
devops.git>
```

Переходить у директорію чарта, де лежить `values.yaml`.

```
cd goit-devops/django-chart
```

Замінює рядок `tag: ...` на новий тег, наприклад `tag: v1.0.42`.

```
sed -i "s/tag: .*/tag: $IMAGE_TAG/" values.yaml
```

Налаштовує Git-автора (щоб коміт був валідним і мав зрозумілу історію).

```
git config user.email "$COMMIT_EMAIL"
git config user.name "$COMMIT_NAME"
```

Додає змінений файл у Git, комітить із повідомленням про новий тег і пушить прямо в гілку `main`.

```
git add values.yaml
git commit -m "Update image tag to $IMAGE_TAG"
git push origin main
```

Глянемо, що змінилося в нашому репозиторії з інфраструктурою в директорії `django-chart/values.yaml`:

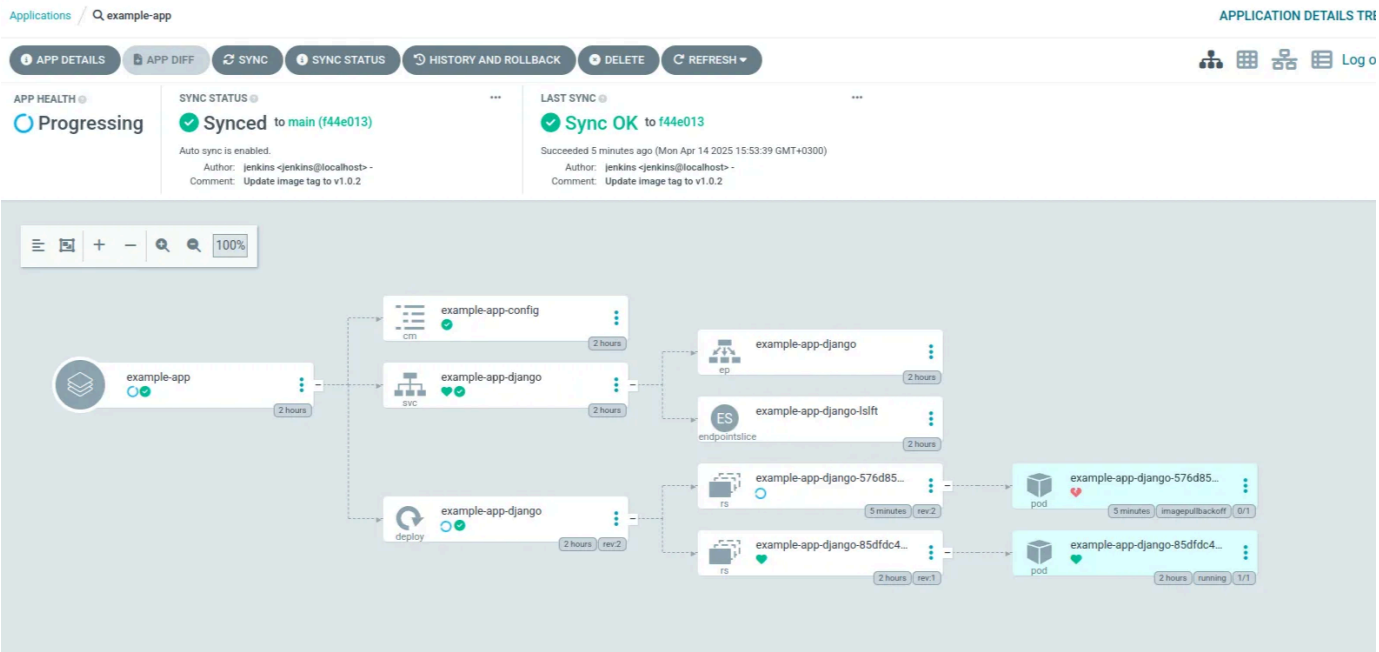
goit-devops / django-chart / values.yaml

jenkins Update image tag to v1.0.2

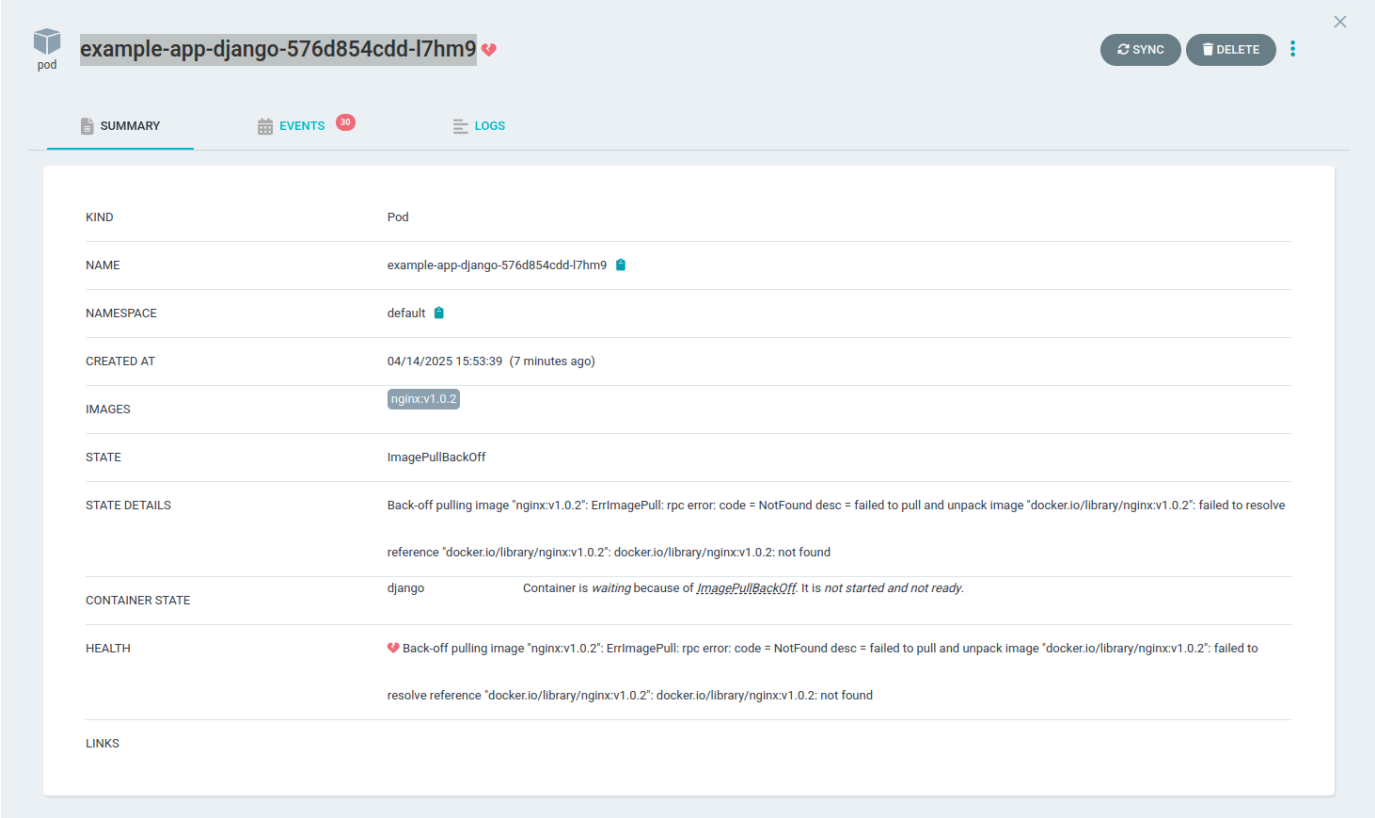
Code Blame 18 lines (15 loc) · 271 Bytes

```yaml
image:
  repository: nginx
  tag: v1.0.2
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false

config:
  POSTGRES_PORT: 5432
  POSTGRES_HOST: db
  POSTGRES_USER: django_user
  POSTGRES_DB: django_db
  POSTGRES_PASSWORD: pass9764gd
```

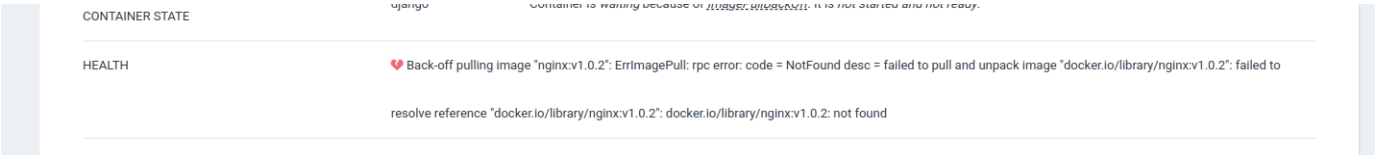
Отже, після того як ми оновили Helm chart і задали новий тег `v1.0.2`, ArgoCD автоматично підхопив цю зміну й ініціював оновлення застосунку



👆 В інтерфейсі ArgoCD ми можемо бачити, що оновлений `Deployment` створив новий Pod `example-app-django-576d854cdd-l7hm9`, який, однак, не зміг стартувати.



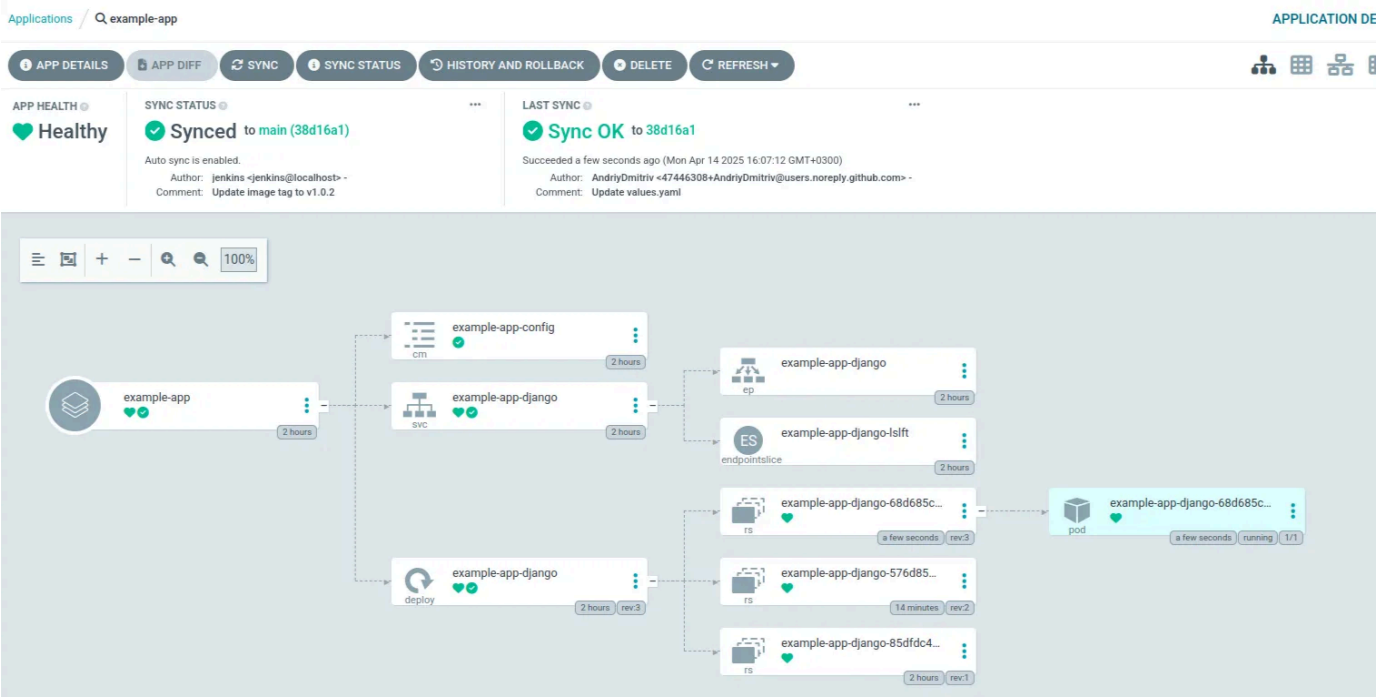
Це очікувана ситуація, оскільки в Helm chart ми навмисно залишили `image.repository: nginx`, а тег `v1.0.2` не існує в офіційному репозиторії образів nginx. Через це Kubernetes не може знайти такий образ у Docker Hub, і Pod переходить у стан `ImagePullBackOff` 🤷

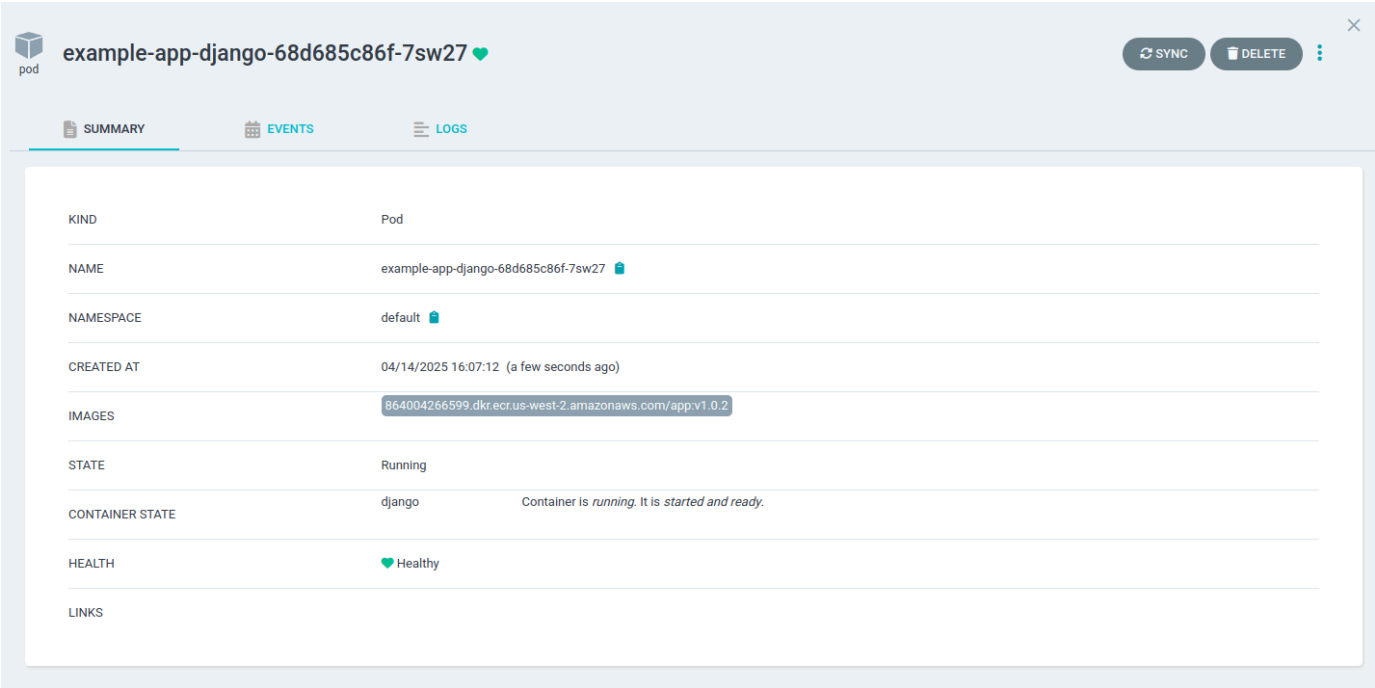


Цей приклад добре ілюструє, як ArgoCD дозволяє не лише автоматично деплоїти зміни, але й миттєво **помічати помилки розгортання** через візуальний інтерфейс.

Завдяки цьому ви можете швидко локалізувати проблему, наприклад помилку в образі, конфігурації або відсутність потрібного тегу.

А зараз давайте змінимо на валідні значення й повторимо ті кроки, які зазначено вище:





Усе тепер працює як годинник.

Цей етап автоматично оновлює Helm chart після кожного білду, щоб Argo CD (або будь-який GitOps-оператор) міг підхопити нову версію Docker-образу та задеплоїти її у кластер.